

TD – Algorithmique

Apprendre et Maîtriser les Bases

avec AlgoBox

Cours : Introduction à l'algorithmique

Outil : AlgoBox (logiciel gratuit)

Niveau : Débutant – IUT / BTS / Licence

Objectifs du TD :

- Comprendre les concepts fondamentaux de l'algorithmique
- Maîtriser l'interface et la syntaxe d'AlgoBox
- Savoir écrire, tester et déboguer des algorithmes
- Pratiquer les structures de base : variables, conditions, boucles, tableaux, fonctions

Table des matières

1	Introduction à AlgoBox	2
2	Variables et affectation	4
3	Entrées et sorties	5
4	Structures conditionnelles	7
5	Structures itératives (boucles)	9
6	Tableaux	11
7	Fonctions	13
8	Exercices de synthèse	14
9	Barème de notation	16

1 Introduction à AlgoBox

Rappel Qu'est-ce qu'AlgoBox ?

AlgoBox est un logiciel libre et gratuit d'aide à l'algorithmique. Il permet d'écrire, d'exécuter et de déboguer des algorithmes en pseudo-code français. Il est conçu pour l'enseignement de l'algorithmique au lycée, en IUT et en BTS.

Téléchargement : <http://www.xm1math.net/algobox/>

Rappel Structure générale d'un algorithme AlgoBox

```
VARIABLES
  n EST_DU_TYPE NOMBRE
  ch EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
  // Corps de l'algorithme
FIN_ALGORITHME
```

Un algorithme AlgoBox comporte toujours :

1. Une section **VARIABLES** pour déclarer les variables
2. Un bloc **DEBUT_ALGORITHME** / **FIN_ALGORITHME**

Exercice 1.1 – Découverte de l'interface

Lancez AlgoBox et identifiez les différentes zones de l'interface :

1. Repérez la zone de déclaration des variables.
2. Repérez la zone d'écriture de l'algorithme.
3. Repérez les boutons *Nouveau*, *Ouvrir*, *Tester*, *Pas à pas*.
4. Saisissez l'algorithme suivant et testez-le :

```
VARIABLES
  prenom EST_DU_TYPE CHAINE
DEBUT_ALGORITHME
  LIRE prenom
  AFFICHER "Bonjour "
  AFFICHER prenom
  AFFICHER " !"
FIN_ALGORITHME
```

5. Que se passe-t-il lorsque vous cliquez sur *Tester* ? Décrivez le déroulement.
6. Utilisez le mode *Pas à pas*. Qu'observez-vous ?

Exercice 1.2 – Premier algorithme : calcul simple

Écrivez un algorithme AlgoBox qui :

1. Demande à l'utilisateur de saisir deux nombres **a** et **b**.

2. Calcule et affiche leur somme, leur différence, leur produit et leur quotient.

Conseil

Pensez à déclarer les variables nécessaires. Pour le quotient, que se passe-t-il si b vaut 0 ?

Exercice 1.3 – Types de variables

Pour chaque situation, indiquez le type AlgoBox le plus adapté (NOMBRE, CHAÎNE ou BOOLEEN) :

1. L'âge d'une personne
2. Le nom d'une ville
3. Le fait qu'un nombre soit pair ou impair
4. La température extérieure
5. Un numéro de téléphone
6. La réponse à la question *Est-ce que $a > b$?*
7. La moyenne générale d'un élève
8. Le code postal

Justifiez chaque choix.

Exercice 1.4 – Affectation et trace

Considérez l'algorithme suivant :

```
VARIABLES
a EST_DU_TYPE NOMBRE
b EST_DU_TYPE NOMBRE
c EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
a PREND_LA_VALEUR 3
b PREND_LA_VALEUR 5
c PREND_LA_VALEUR a + b
a PREND_LA_VALEUR c - a
b PREND_LA_VALEUR a + 2
AFFICHER "a = "
AFFICHER a
AFFICHER "b = "
AFFICHER b
AFFICHER "c = "
AFFICHER c
FIN_ALGORITHME
```

1. Complétez le tableau de trace ci-dessous :

Instruction	a	b	c
$a \leftarrow 3$	3	–	–
$b \leftarrow 5$	3	5	–
$c \leftarrow a+b$			
$a \leftarrow c-a$			
$b \leftarrow a+2$			

2. Quelles valeurs seront affichées par l'algorithme ?
3. Vérifiez en exécutant l'algorithme dans AlgoBox.

2 Variables et affectation

Rappel Règles fondamentales

- Une variable est un *contenant* : elle possède un **nom**, un **type** et une **valeur**.
- L'**affectation** $\text{var} \leftarrow \text{expression}$ *écrase* l'ancienne valeur de **var**.
- En AlgoBox : `var PREND_LA_VALEUR expression`
- L'ordre des instructions est **séquentiel** : chaque instruction s'exécute l'une après l'autre.

Exercice 2.1 – Échange de deux variables

On dispose de deux variables **x** et **y** contenant respectivement les valeurs 7 et 12.

1. Proposez un algorithme qui échange les valeurs de **x** et **y** (après exécution, **x** vaut 12 et **y** vaut 7). Vous pourrez utiliser une variable temporaire.
2. Peut-on faire l'échange sans variable temporaire ? Si oui, comment ?

Exercice 2.2 – Affectations enchaînées

Donnez les valeurs finales de **a**, **b**, **c** après l'exécution de chaque séquence :

Séquence 1 :

```
a PREND_LA_VALEUR 2
b PREND_LA_VALEUR a + 3
a PREND_LA_VALEUR a + b
c PREND_LA_VALEUR a * b
```

Séquence 2 :

```
a PREND_LA_VALEUR 10
b PREND_LA_VALEUR a - 4
a PREND_LA_VALEUR b
c PREND_LA_VALEUR a + b + 1
```

Séquence 3 :

```
a PREND_LA_VALEUR 1
b PREND_LA_VALEUR 2
c PREND_LA_VALEUR 3
a PREND_LA_VALEUR b
```

```
b PREND_LA_VALEUR c
c PREND_LA_VALEUR a
```

Exercice 2.3 – Calculs mathématiques

Écrivez un algorithme AlgoBox qui :

1. Demande à l'utilisateur un nombre réel x .
2. Calcule et affiche x^2 , x^3 , $\sqrt{x}$, et $\frac{1}{x}$.

Conseil

En AlgoBox, la puissance s'écrit `pow(x,n)` pour x^n , et la racine carrée s'écrit `sqrt(x)`.

Exercice 2.4 – Conversion d'unités

Écrivez un algorithme qui :

1. Demande une température en degrés Celsius.
2. La convertit en Fahrenheit selon la formule : $F = \frac{9}{5} \times C + 32$.
3. Affiche le résultat avec un message explicatif.

Puis écrire l'algorithme inverse : de Fahrenheit vers Celsius ($C = \frac{5}{9} \times (F - 32)$).

Exercice 2.5 – Aires et périmètres

Écrivez des algorithmes AlgoBox pour calculer et afficher :

1. L'aire et le périmètre d'un rectangle (largeur et longueur saisies).
2. L'aire et la circonférence d'un cercle (rayon saisi).
3. L'aire d'un triangle (base et hauteur saisies).

Rappel : $\pi \approx 3,14159$. En AlgoBox, on utilise `PI`.

3 Entrées et sorties

Rappel Instructions de base

- `LIRE var` : attend que l'utilisateur saisisse une valeur et la stocke dans `var`.
- `AFFICHER expression` : affiche le résultat sans retour à la ligne.
- `AFFICHER* expression` : affiche le résultat avec retour à la ligne.
- `AFFICHER "texte"` : affiche un message littéral.

Exercice 3.1 – Affichage formaté

Écrivez un algorithme qui demande le prénom, l'âge et la ville de l'utilisateur, puis affiche :
Bonjour, je m'appelle <prenom>, j'ai <age> ans et j'habite <ville>.
Utilisez correctement AFFICHER et AFFICHER* pour obtenir un affichage sur une seule ligne.

Exercice 3.2 – Calcul du prix TTC

Écrivez un algorithme qui :

1. Demande le prix hors taxes (HT) d'un article.
2. Demande le taux de TVA (en pourcentage, par exemple 20 pour 20%).
3. Calcule le prix TTC : $TTC = HT \times (1 + \frac{TVA}{100})$
4. Affiche le résultat sous la forme : Prix HT : ... € -- TVA : ...% -- Prix TTC : ... €

Exercice 3.3 – Moyenne de notes

Écrivez un algorithme qui :

1. Demande à l'utilisateur de saisir 3 notes (sur 20).
2. Calcule la moyenne.
3. Affiche chaque note et la moyenne avec un message clair.
4. Affiche si la moyenne est supérieure ou égale à 10 (mention *passable*) ou inférieure.

Exercice 3.4 – Simulation de sortie AlgoBox

Pour l'algorithme ci-dessous, simulez l'affichage produit si l'utilisateur saisit successivement les valeurs 5, 3 et 8 :

```
VARIABLES
a EST_DU_TYPE NOMBRE
b EST_DU_TYPE NOMBRE
c EST_DU_TYPE NOMBRE
s EST_DU_TYPE NOMBRE
m EST_DU_TYPE NOMBRE
DEBUT_ALGORITHME
  LIRE a
  LIRE b
  LIRE c
  s PREND_LA_VALEUR a+b+c
  m PREND_LA_VALEUR s/3
  AFFICHER "Somme = "
  AFFICHER* s
  AFFICHER "Moyenne = "
  AFFICHER* m
FIN_ALGORITHME
```

Recopiez l'écran de sortie tel qu'il apparaîtra dans AlgoBox.

4 Structures conditionnelles

Rappel Syntaxe AlgoBox

Condition simple :

```
SI (condition) ALORS
  DEBUT_SI
    // instructions
  FIN_SI
```

Condition avec alternative :

```
SI (condition) ALORS
  DEBUT_SI
    // instructions si VRAI
  FIN_SI
SINON
  DEBUT_SINON
    // instructions si FAUX
  FIN_SINON
```

Opérateurs de comparaison : ==, !=, <, >, <=, >=

Opérateurs logiques : ET, OU, NON

Exercice 4.1 – Pair ou impair

Écrivez un algorithme qui :

1. Demande un entier n .
2. Détermine si n est pair ou impair en utilisant l'opérateur modulo (MOD).
3. Affiche "<n> est pair" ou "<n> est impair".

Conseil

$n \text{ MOD } 2$ donne le reste de la division de n par 2. Si ce reste vaut 0, le nombre est pair.

Exercice 4.2 – Maximum de deux nombres

Écrivez un algorithme qui :

1. Demande deux nombres a et b .
2. Affiche le plus grand des deux avec un message : "Le maximum est : ..."

Exercice 4.3 – Maximum de trois nombres

Écrivez un algorithme qui détermine le maximum de trois nombres a , b , c saisis par l'utilisateur.

Contrainte : n'utilisez pas de variable supplémentaire (hormis a , b , c et une variable **max**).

Exercice 4.4 – Année bissextile

Une année est bissextile si :

- elle est divisible par 4, **MAIS PAS** divisible par 100,
- **OU** elle est divisible par 400.

Écrivez un algorithme qui demande une année et affiche si elle est bissextile ou non.

Exercice 4.5 – Calcul du impôt simplifié

On considère le barème simplifié suivant :

Revenu imposable R	Taux
$R \leq 10\,084$ €	0%
$10\,084 < R \leq 25\,710$ €	11%
$25\,710 < R \leq 73\,516$ €	30%
$R > 73\,516$ €	41%

Écrivez un algorithme qui demande le revenu imposable et affiche le montant de l'impôt.

Exercice 4.6 – Mention du baccalauréat

Écrivez un algorithme qui demande la moyenne générale d'un élève (sur 20) et affiche la mention :

Moyenne m	Mention
$m < 8$	Recalé
$8 \leq m < 10$	Rattrapage
$10 \leq m < 12$	Passable
$12 \leq m < 14$	Assez bien
$14 \leq m < 16$	Bien
$m \geq 16$	Très bien

Utilisez des SI ... ALORS ... SINON imbriqués.

Exercice 4.7 – Opérations sur les chaînes

Écrivez un algorithme qui :

1. Demande un mot à l'utilisateur.
2. Vérifie si le mot commence par une voyelle ou une consonne.
3. Affiche le résultat.

Conseil

En AlgoBox, `mot.substring(0,1)` retourne le premier caractère d'une chaîne `mot`. On peut tester si ce caractère appartient à l'ensemble {a, e, i, o, u, y}.

5 Structures itératives (boucles)

Rappel Les trois types de boucles en AlgoBox

Boucle POUR (nombre d'itérations connu) :

```
POUR i ALLANT_DE 1 A n
  DEBUT_POUR
    // instructions
  FIN_POUR
```

Boucle TANT QUE (condition d'entrée) :

```
TANT_QUE (condition) FAIRE
  DEBUT_TANT_QUE
    // instructions
  FIN_TANT_QUE
```

Règle importante : la boucle TANT_QUE teste la condition **avant** chaque itération. Si la condition est fausse dès le départ, le corps n'est jamais exécuté.

Exercice 5.1 – Somme des n premiers entiers

Écrivez un algorithme qui :

1. Demande un entier n ($n > 0$).
2. Calcule la somme $S = 1 + 2 + 3 + \dots + n$ en utilisant une boucle POUR.
3. Affiche le résultat.
4. Vérifiez avec la formule $S = \frac{n(n+1)}{2}$.

Exercice 5.2 – Table de multiplication

Écrivez un algorithme qui :

1. Demande un entier n .
2. Affiche la table de multiplication de n :
 $1 \times n = \dots, 2 \times n = \dots, \dots, 10 \times n = \dots$

Exercice 5.3 – Factorielle

Écrivez un algorithme qui :

1. Demande un entier n ($n \geq 0$).
2. Calcule $n! = 1 \times 2 \times 3 \times \dots \times n$ (avec $0! = 1$).
3. Affiche le résultat.

Utilisez d'abord une boucle POUR, puis réécrivez l'algorithme avec une boucle TANT_QUE.

Exercice 5.4 – Puissance

Écrivez un algorithme qui :

1. Demande un réel x et un entier $n \geq 0$.
2. Calcule x^n sans utiliser la fonction `pow()`.
3. Affiche le résultat.

Exercice 5.5 – Deviner un nombre

L'ordinateur choisit un nombre aléatoire entre 1 et 100. Le joueur doit le deviner.
Écrivez un algorithme qui :

1. Initialise un nombre secret avec `ALGOBOX_RANDOM_INTEGER(1,100)`.
2. Demande répétitivement à l'utilisateur de proposer un nombre.
3. Après chaque proposition, affiche *Trop grand*, *Trop petit* ou *Bravo !*.
4. Compte le nombre d'essais et l'affiche à la fin.

Exercice 5.6 – Suite de Fibonacci

La suite de Fibonacci est définie par : $u_0 = 0$, $u_1 = 1$, et $u_n = u_{n-1} + u_{n-2}$ pour $n \geq 2$.
Écrivez un algorithme qui :

1. Demande un entier n .
2. Calcule et affiche les $n + 1$ premiers termes de la suite (de u_0 à u_n).

Exercice 5.7 – Plus grand commun diviseur (PGCD)

Écrivez un algorithme qui calcule le PGCD de deux entiers a et b par la méthode d'Euclide :

1. Tant que $b \neq 0$, remplacer (a, b) par $(b, a \bmod b)$.
2. Le PGCD est la dernière valeur non nulle de a .

Affiche le résultat sous la forme : "PGCD de $\langle a \rangle$ et $\langle b \rangle$ = $\langle \text{resultat} \rangle$ ".

Exercice 5.8 – Somme des chiffres

Écrivez un algorithme qui :

1. Demande un entier positif n .
2. Calcule la somme de ses chiffres. Par exemple, pour $n = 1234$, la somme est $1 + 2 + 3 + 4 = 10$.

Conseil

Utilisez les opérations `MOD 10` (pour obtenir le dernier chiffre) et la division entière `/` (en AlgoBox, `n/10` avec `n` entier tronque la partie décimale).

Exercice 5.9 – Nombre premier

Écrivez un algorithme qui :

1. Demande un entier $n > 1$.
2. Détermine si n est premier en testant les diviseurs de 2 à $\sqrt{n}$.
3. Affiche "<n> est premier" ou "<n> n'est pas premier".

Conseil

Un nombre est premier s'il n'a pas de diviseur autre que 1 et lui-même. Il suffit de tester jusqu'à $\lfloor \sqrt{n} \rfloor$ car si $n = d \times k$, l'un des deux est $\leq \sqrt{n}$.

6 Tableaux**Rappel Déclaration et utilisation**

En AlgoBox, les tableaux se déclarent et s'utilisent ainsi :

Déclaration :

```
monTableau EST_DU_TYPE LISTE
```

Affectation : `monTableau[indice] ← valeur`

En AlgoBox : `monTableau[indice] PREND_LA_VALEUR valeur`

Accès : `AFFICHER monTableau[indice]`

Taille : La taille est définie lors de la première utilisation ou par `ALGOBOX_DEFAULT_LISTE`.

Exercice 6.1 – Remplissage et affichage d'un tableau

Écrivez un algorithme qui :

1. Demande à l'utilisateur la taille n du tableau.
2. Remplit le tableau avec n valeurs saisies par l'utilisateur.
3. Affiche toutes les valeurs du tableau.

Exercice 6.2 – Somme et moyenne d'un tableau

À partir du tableau rempli dans l'exercice précédent, écrire un algorithme qui :

1. Calcule la somme de tous les éléments.
2. Calcule la moyenne.
3. Affiche les deux résultats.

Exercice 6.3 – Recherche du maximum et du minimum

Écrivez un algorithme qui :

1. Remplit un tableau de n nombres.
2. Recherche et affiche le maximum et le minimum du tableau.
3. Affiche aussi les indices correspondants.

Exercice 6.4 – Recherche d'une valeur

Écrivez un algorithme qui :

1. Remplit un tableau de n entiers.
2. Demande une valeur v à rechercher.
3. Parcourt le tableau et affiche *Valeur trouvée à l'indice ...* ou *Valeur non trouvée*.
4. Compte le nombre d'occurrences de v dans le tableau.

Exercice 6.5 – Tri par sélection

Implémentez l'algorithme de tri par sélection :

1. Pour chaque position i de 0 à $n - 2$:
 - (a) Chercher le minimum dans le sous-tableau $[i..n - 1]$.
 - (b) Échanger ce minimum avec l'élément à la position i .
2. Afficher le tableau trié.

Exercice 6.6 – Tri à bulles

Implémentez l'algorithme de tri à bulles :

1. Parcourir le tableau en comparant les éléments adjacents.
2. Si deux éléments consécutifs sont dans le mauvais ordre, les échanger.
3. Répéter jusqu'à ce qu'aucun échange ne soit nécessaire.
4. Afficher le tableau trié et le nombre de passages effectués.

Exercice 6.7 – Comptage de fréquences

Écrivez un algorithme qui :

1. Remplit un tableau de n entiers compris entre 0 et 20 (notes).
2. Compte combien de fois chaque note apparaît.
3. Affiche l'histogramme des fréquences :
Note 0 : *** (3 occurrences)

Note 1 : * (1 occurrence)
...

Utilisez un tableau de 21 cases (indices 0 à 20) pour stocker les compteurs.

7 Fonctions

Rappel Définition et appel de fonction

Définition :

```
FONCTIONS_UTILISEES :  
  FONCTION maFonction(param1, param2)  
    DEBUT_FONCTION  
      // corps de la fonction  
    RETOURNER resultat  
  FIN_FONCTION
```

Appel : maFonction(arg1, arg2)

Les fonctions permettent de **décomposer** un problème en sous-problèmes réutilisables.

Exercice 7.1 – Fonction carré

Écrivez une fonction **carré(x)** qui retourne x^2 . Utilisez-la dans un algorithme qui :

1. Demande un nombre x .
2. Affiche x^2 en appelant la fonction.

Exercice 7.2 – Fonction estPair

Écrivez une fonction **estPair(n)** qui retourne **VRAI** si n est pair, **FAUX** sinon.

Utilisez-la dans un algorithme qui demande 5 nombres et affiche pour chacun s'il est pair ou impair.

Exercice 7.3 – Fonction maximum

Écrivez une fonction **max(a, b)** qui retourne le plus grand de deux nombres.

Utilisez-la pour déterminer le maximum de trois nombres en appelant **max(max(a,b), c)**.

Exercice 7.4 – Fonction factorielle

Écrivez une fonction récursive **factorielle(n)** :

1. Si $n = 0$, retourner 1.
2. Sinon, retourner $n \times \text{factorielle}(n - 1)$.

Conseil

AlgoBox ne supporte pas nativement la récursivité. Implémentez plutôt la version itérative avec une boucle.

Exercice 7.5 – Fonction estPremier

Écrivez une fonction `estPremier(n)` qui retourne **VRAI** si n est premier, **FAUX** sinon. Utilisez-la dans un algorithme qui :

1. Demande deux entiers a et b .
2. Affiche tous les nombres premiers compris entre a et b .

Exercice 7.6 – Fonction PGCD

Écrivez une fonction `pgcd(a, b)` qui retourne le PGCD de deux entiers par l'algorithme d'Euclide. Utilisez-la pour simplifier une fraction $\frac{a}{b}$: divisez a et b par leur PGCD et affichez la fraction simplifiée.

8 Exercices de synthèse**Exercice 8.1 – Calculatrice simple**

Écrivez un algorithme qui simule une calculatrice :

1. Demande deux nombres a et b .
2. Demande l'opération souhaitée (+, -, *, /).
3. Affiche le résultat.
4. Gère le cas de la division par zéro.
5. Demande si l'utilisateur veut effectuer un autre calcul (boucle principale).

Exercice 8.2 – Jeu du pendu simplifié

Écrivez un algorithme qui :

1. Choisisse un mot aléatoirement parmi une liste prédéfinie de 5 mots.
2. Affiche le mot avec des tirets (ex : "- - - - -" pour un mot de 5 lettres).
3. Demande répétitivement une lettre à l'utilisateur.
4. Révèle les lettres trouvées.
5. Limite le nombre d'essais à 8.
6. Affiche *Gagné* ou *Perdu*.

Exercice 8.3 – Statistiques de notes

Écrivez un algorithme complet de gestion de notes :

1. Saisir le nombre d'élèves puis les notes de chaque élève.

2. Calculer et afficher la moyenne de la classe.
3. Calculer et afficher l'écart-type : $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (note_i - moy)^2}$
4. Afficher le nombre de notes au-dessus et en dessous de la moyenne.
5. Afficher l'histogramme simplifié des mentions.

Exercice 8.4 – Chiffrement de César

Le chiffrement de César consiste à décaler chaque lettre d'un message d'un nombre fixe de positions dans l'alphabet.

Écrivez un algorithme qui :

1. Demande un message (chaîne de caractères) et une clé (décalage entier).
2. Chiffre le message en décalant chaque lettre de la clé.
3. Affiche le message chiffré.

Conseil

En AlgoBox, la fonction `caractereVersCode(c)` donne le code ASCII d'un caractère, et `codeVersCaractere(n)` donne le caractère correspondant. Le code de 'A' est 65.

Exercice 8.5 – Conversions numériques

Écrivez deux algorithmes :

Algorithme 1 : Conversion décimal → binaire

1. Demande un entier positif n .
2. Convertit n en binaire par divisions successives par 2.
3. Affiche la représentation binaire.

Algorithme 2 : Conversion binaire → décimal

1. Demande une chaîne binaire (ex : "10110").
2. Convertit en décimal en utilisant les puissances de 2.
3. Affiche la valeur décimale.

9 Barème de notation

Chapitre	Exercices	Points
1 – Introduction à AlgoBox	1.1, 1.2, 1.3, 1.4	3
2 – Variables et affectation	2.1 à 2.5	3
3 – Entrées et sorties	3.1 à 3.4	2
4 – Structures conditionnelles	4.1 à 4.7	4
5 – Boucles	5.1 à 5.9	4
6 – Tableaux	6.1 à 6.7	3
7 – Fonctions	7.1 à 7.6	3
8 – Synthèse	8.1 à 8.5	2
Total		20

Conseil

Conseils généraux :

- Testez **toujours** vos algorithmes dans AlgoBox avant de les rendre.
- Utilisez le mode *Pas à pas* pour déboguer.
- Vérifiez les cas limites (division par zéro, valeur négative, chaîne vide).
- Commentez votre code pour faciliter la relecture.